

Chunking, Retrieval, and RAG Notes

Ingestion, chunks, overlap, retrieval, grounding, unsupported questions, citations, and RAG failure modes.

Engineering notes and worked references

Contents

1. Document ingestion
2. Chunk size
3. Overlap
4. Natural section boundaries
5. Retrieval query
6. Retrieval and reranking
7. Grounded answer prompts
8. Unsupported questions
9. Retrieval failure vs answer failure
10. Citations and source display
11. Evaluating a small RAG system

1. Document ingestion

A RAG system first turns source documents into text units that can be searched. Extraction quality matters because the retriever cannot recover text that was never captured correctly.

Keep source identifiers and section boundaries during ingestion. They are useful for citations, debugging, and rebuilding the index.

Example from use

A PDF heading can become metadata while the paragraph text becomes chunk content.

If a table is important, make sure its extracted text remains understandable.

Things people mix up

- Dropping filenames after extraction.
- Indexing empty headers as if they were content.
- Ignoring extraction errors because the PDF opens visually.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of document ingestion becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

2. Chunk size

Chunking splits long text into searchable pieces. Small chunks are precise but can lose context. Large chunks preserve context but can mix several topics.

A useful chunk is usually large enough to answer one coherent question and small enough that unrelated sections do not dominate its embedding.

Example from use

A short section on 'top-k' can stay together.

A long handbook chapter may need several chunks with section metadata.

Things people mix up

- Picking a chunk size only because another tutorial used it.
- Counting characters and words as if they were tokens.
- Splitting code examples halfway through without context.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of chunk size becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

3. Overlap

Overlap repeats a small amount of text between neighboring chunks so a sentence near a boundary is not isolated from its supporting context.

Overlap is helpful when ideas span boundaries, but too much overlap creates many near-duplicate results.

Example from use

A 400-word chunk with modest overlap can preserve a definition and its example.

If every returned result contains the same paragraph, overlap may be too high.

Things people mix up

- Using 80 percent overlap by default.
- Assuming overlap fixes a bad document structure.
- Counting duplicates as independent evidence.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of overlap becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

4. Natural section boundaries

Headings, paragraphs, lists, and code blocks can provide better chunk boundaries than a blind fixed window.

Natural boundaries help keep a rule with its explanation. They can also make chunk sizes uneven, which is usually acceptable if the units remain useful.

Example from use

Keep a short 'Error handling' section intact instead of combining it with an unrelated section just to reach a target size.

A table can be stored with the paragraph that explains it.

Things people mix up

- Making every chunk exactly the same length at the cost of meaning.
- Dropping headings that carry the subject of the paragraph.
- Creating tiny heading-only chunks.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of natural section boundaries becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

5. Retrieval query

The text sent to the vector search is not always identical to the raw user message. A standalone direct question usually works as-is. A vague follow-up often needs rewriting first.

The retrieval query should contain enough subject context to match the source without turning into a long summary of the whole conversation.

Example from use

'What about top-k?' after discussing vector retrieval can become 'How does top-k affect vector retrieval?'.

'Now write a Python loop' is a new topic and should not be rewritten into the previous RAG discussion.

Things people mix up

- Stuffing the full history into the vector query.
- Forcing old context into a topic change.
- Rewriting direct questions that were already clear into something less precise.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of retrieval query becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?

- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

6. Retrieval and reranking

The first retrieval step returns likely chunks. A small application may use those results directly. Larger systems sometimes apply a second ranking step.

The important part is to know which chunks are being passed forward and why. A more complex ranking pipeline is not automatically better.

Example from use

For a small note library, cosine similarity plus metadata may be enough.

If exact identifiers are common, combine semantic retrieval with an exact check.

Things people mix up

- Adding a reranker before measuring the baseline.
- Hiding retrieval decisions so developers cannot inspect them.
- Using ranking complexity to compensate for poor source text.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of retrieval and reranking becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

7. Grounded answer prompts

The answer step should clearly separate instructions from retrieved source text. Tell the model to use the supplied material and what to do when it does not support the answer.

Grounding is a product behavior, not magic. The model can still ignore a passage, so evaluation and source display remain important.

Example from use

A useful instruction is 'Answer using the notes below. If they do not cover the question, say so.'

Keep source labels near the retrieved text so the answer can cite them.

Things people mix up

- Saying 'never hallucinate' and assuming the problem is solved.
- Mixing retrieved text with system instructions without labels.
- Asking for citations when chunk metadata does not contain source information.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of grounded answer prompts becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

8. Unsupported questions

A source-bound assistant should have a clear behavior when the notes do not answer the question. It can say the material does not cover it and optionally mention what related material was found.

Do not answer from general model knowledge when the product promise is 'answer from the notes' unless the user explicitly asks for outside knowledge.

Example from use

A question about a Kubernetes feature should remain unsupported if the supplied notes only cover FastAPI, SQLite, and RAG.

A weak related match should not be stretched into a confident answer.

Things people mix up

- Using the model's background knowledge silently.
- Returning 'I don't know' when the notes actually contain the answer under different wording.
- Treating no exact keyword match as unsupported without semantic retrieval.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of unsupported questions becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

9. Retrieval failure vs answer failure

These two failures look similar to the user but need different fixes. Retrieval failure means the needed source never reaches the model. Answer failure means the needed source is present but the generated answer still misses or contradicts it.

Always inspect the retrieved chunks before changing prompts or models.

Example from use

If the correct chunk is absent, check query rewrite, chunking, embeddings, filters, and k.

If the correct chunk is present, check answer instructions, context ordering, and output validation.

Things people mix up

- Calling every wrong answer a hallucination problem.
- Tuning temperature for a missing retrieval result.
- Changing the vector store because the generator ignored correct context.

Quick check

- Can the idea be explained without relying on one memorized definition?

- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of retrieval failure vs answer failure becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

10. Citations and source display

A source reference should let a reviewer locate the material behind the answer. File name plus section or chunk ID is often enough for a small system.

Citations are easier when chunk metadata is preserved from ingestion. Do not generate source names from memory.

Example from use

Return `source_file='03_chunking_retrieval_and_rag_notes.pdf', section='Top-k', chunk_id='rag-08'`.

A debug view can show the chunk text and score separately from the final answer.

Things people mix up

- Inventing page numbers that were not stored.
- Citing a document that was not retrieved.
- Returning only a score with no source identifier.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of citations and source display becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?
- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.

11. Evaluating a small RAG system

A useful test set includes direct questions, paraphrases, vague follow-ups, multi-section questions, and unsupported questions.

The goal is to observe whether retrieval finds the right source and whether the answer uses it correctly. A small manually reviewed set is better than no evaluation.

Example from use

Record expected source sections for a few important questions.

Include a question where top-1 is not enough but top-3 is.

Things people mix up

- Testing only the exact wording used in the notes.
- Judging quality only by whether the answer sounds fluent.
- Skipping unsupported questions.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

End-to-end view

The value of evaluating a small rag system becomes clearer when the whole path is visible: source text, chunk, search query, retrieved result, answer prompt, and final response. Each stage can be correct or wrong independently.

For a small note assistant, keeping a trace of these stages is more useful than adding a complicated evaluation framework. A few saved test questions with expected source sections can reveal whether a later code change improved or damaged retrieval.

Questions worth testing

- Does a paraphrase find the same source as the wording used in the notes?
- Does a vague follow-up retrieve the right section after rewriting?
- Does an unsupported question stay unsupported?

- Can the answer show the exact document and section it used?

When an answer is poor, avoid changing chunking, embeddings, top-k, prompt wording, and model choice all at once. Change one layer, rerun the same question, and compare the trace.